

CS5013 Project Proposal

Automated Java-Based MRI Volume Normalizer and Bi-Directional Converter

21 August 2026

1 Team information

Team ID: Solo

•

2 Problem Statement

Research MRI datasets frequently arrive as heterogeneous collections of files distributed across deeply nested directories, with inconsistent naming conventions, missing extensions, duplicate filenames, and mixtures of DICOM image instances and NIfTI volumes. Before downstream analysis, researchers may need to inspect directories manually, identify the underlying image format, group DICOM instances into acquisitions, reorganize files, and convert between formats.

This project proposes a standalone Java 21 desktop application that automates this pre-processing workflow. The system will recursively inspect an input directory, identify likely DICOM and NIfTI files from their binary structure rather than their filenames, group DICOM instances into coherent series, flatten or reorganize the source hierarchy into a collision-safe directory structure, and perform supported DICOM-to-NIfTI and NIfTI-to-DICOM conversions.

The important scope boundary is that “bi-directional conversion” will mean preservation of image data and the spatial metadata that can be represented by both formats within a documented supported subset; it will *not* claim byte-for-byte reversibility of arbitrary DICOM objects. NIfTI is deliberately a simpler image-exchange format and cannot represent the complete set of DICOM attributes, private tags, acquisition details, or object semantics of an arbitrary DICOM dataset. The application will therefore maintain a conversion/provenance manifest for metadata that cannot be represented directly in NIfTI.

3 Users and Stakeholder

Named stakeholder: Dr. Rupsa Bhattacharjee, Assistant Professor & PI CBI-HI Lab, Department of Medical Sciences and Technology (MST), IIT Madras.

I consulted with Dr. Rupsa Bhattacharjee on 17 August 2026 to address computational bottlenecks in handling cohort MRI datasets. She identified unpredictable directory structures, mixed or missing file extensions, and repetitive manual inspection and conversion as practical pre-processing burdens in research workflows. The proposed tool is intended to provide a local, self-contained desktop interface that can scan research datasets, organize imaging files, validate their underlying formats, and perform controlled conversions without requiring a separate Python environment or command-line workflow.

The stakeholder's acknowledgement is included in Appendix A.

4 Prior Work and Why It Does Not Fully Suffice

4.1 dcm2niix

dcm2niix is a mature research converter designed primarily to convert DICOM into NIfTI and can handle complex vendor-specific cases. It exposes a command-line workflow and also produces useful metadata sidecars. Its scope is not the same as the proposed end-user workflow: the proposed application combines recursive dataset discovery, binary-format validation, directory normalization, provenance tracking, a local GUI, and a controlled reverse conversion path in one Java application. The comparison is therefore one of workflow integration rather than claiming that the project will outperform dcm2niix as a DICOM-to-NIfTI converter. [1]

4.2 NiBabel / SimpleITK-based Scripts

Python libraries make image I/O and format conversion accessible, but a lab workflow built around ad-hoc scripts still requires a Python environment and application-specific wrapper code for directory traversal, collision handling, logging, and user interaction. The proposed system instead targets a Java desktop workflow with a fixed application entry point and a GUI.

4.3 ImageJ / Fiji and Manual Export

ImageJ/Fiji can import and export many medical image formats, but manual batch organization and conversion are not a substitute for deterministic dataset discovery, series grouping, collision-safe relocation, provenance logging, and automated validation. The proposed application targets those operational tasks rather than general-purpose image analysis.

5 Tech stack

Language: **Java 21** (course policy).

- Build system: **Apache Maven**
- Framework: **Java Swing + FlatLaf** for the desktop GUI
- Key libraries: **dcm4che-core**, **dcm4che-imageio**, **JUnit 5**, **Java NIO**, and Java's built-in **GZIPInputStream/GZIPOutputStream**
- Where the tool will run: **In CBI-HI Lab workstations** as a standalone executable JAR with the required Maven/runtime dependencies

6 Scope and Non-Goals

The minimum viable implementation will support NIfTI-1 single-file volumes (**.nii** and **.nii.gz**) and a documented subset of conventional DICOM MR image series.

The project will not attempt to support every vendor-specific transfer syntax, every DICOM storage SOP class, DICOM networking/PACS communication, advanced diffusion or spectroscopy metadata reconstruction, arbitrary enhanced multi-frame DICOM objects, or clinical deployment. Such extensions would be appropriate as future work after the core research-dataset workflow is validated.

7 AI-Usage Statement

AI coding assistants such as Claude Code, GitHub Copilot, ChatGPT, or Gemini Pro may be used in accordance with the course policy on AI-assisted programming. Every submitted component will be reviewed and understood by the team, and the team will be able to explain the design, implementation, tests, and limitations during the viva. AI assistance will not replace testing, code review, or technical accountability.

8 Verification and Evaluation Plan

8.1 Test Fixtures

Evaluation will use two complementary fixture sets:

1. A de-identified multi-patient MRI dataset supplied by the stakeholder, containing nested directories, DICOM series, and NIfTI volumes (`.nii` and `.nii.gz`).
2. A synthetic corner-case suite containing wrong/missing extensions, stripped DICOM prefixes where supported by the parser, invalid or truncated files, zero-byte files, duplicate filenames, incompatible DICOM series, GZIP-wrapped NIfTI volumes, and nested folders with repeated directory names.

8.2 Acceptance Criteria

Area	Acceptance criterion
Format detection	100% correct classification on the defined test fixture set, including files whose extensions are incorrect or absent, with ambiguous/corrupt inputs reported rather than silently accepted.
Directory normalization	Every supported input file is either relocated/copied to a deterministic destination or explicitly recorded as skipped/failed; no files are silently overwritten.
DICOM series grouping	Compatible instances are grouped consistently; incompatible dimensions/orientations/acquisitions are not merged into a single volume.
DICOM $\rightarrow$ NIfTI	Output voxel dimensions, datatype, voxel spacing, and spatial orientation are validated against the source series within defined numerical tolerances.
NIfTI $\rightarrow$ DICOM	Generated instances are parseable DICOM objects with valid identifiers, consistent geometry, dimensions, and pixel data.
Round-trip fidelity	For supported round-trip fixtures, voxel arrays and spatial geometry are recovered within predefined tolerances. Metadata that NIfTI cannot represent is checked against the provenance manifest rather than claimed to be preserved inside the NIfTI file.
Usability	A non-programming lab user can select input/output folders, inspect detected files, start a conversion, and identify failures from the GUI without using a shell.

8.3 Quantitative Validation

For numerical validation, the following checks will be performed:

$$\begin{aligned} \text{shape match : } \quad & \mathbf{d}_{\text{out}} = \mathbf{d}_{\text{ref}}, \\ \text{spacing error : } \quad & \max_i |s_i^{\text{out}} - s_i^{\text{ref}}| < \epsilon_s, \\ \text{voxel error : } \quad & \max |I_{\text{out}} - I_{\text{ref}}| = 0 \end{aligned}$$

for integer-preserving conversions where no rescaling is requested. Floating-point data will use a suitable absolute/relative tolerance. Spatial transforms will be compared as affine matrices, and the corresponding voxel-to-world coordinate error will be reported for representative volume corners and centres.

The software will also generate a machine-readable validation report recording input files, detected format, conversion result, dimensions, spacing, affine information, warnings, and failure messages.

9 Milestone Plan

Deadline	Deliverable
11 Sep 2026 – Design Doc	Finalize architecture, format-detection state machine, provenance model, directory normalization design, and DICOM/NIfTI header parsers. Achieve comprehensive unit coverage for the synthetic format/directory fixtures.
9 Oct 2026 – Mid-Demo	Working end-to-end DICOM-to-NIfTI workflow on de-identified stakeholder data, including series grouping, orientation validation, output generation, logs, and initial GUI.
6 Nov 2026 – Final Submission	Integrated Java Swing application, controlled NIfTI-to-DICOM reconstruction, provenance manifest, round-trip validation, stakeholder evaluation, documentation, packaging, and final demonstration.

10 Risks and Mitigation

1. NIfTI cannot represent all DICOM metadata.

Mitigation: Treat NIfTI-to-DICOM as controlled reconstruction. Preserve selected source metadata in a provenance manifest and clearly label generated/default attributes.

2. DICOM vendor and encoding diversity.

Mitigation: Limit the MVP to a documented DICOM MR subset, rely on dcm4che for DICOM parsing/decoding, and reject unsupported transfer syntaxes or structures with explicit diagnostics rather than silently producing incorrect volumes.

3. DICOM files without the Part 10 prefix.

Mitigation: Use the DICM signature as one detection signal but fall back to parser-based DICOM validation. Never treat the presence of the string as the sole definition of DICOM.

4. Geometry errors during series assembly.

Mitigation: Validate dimensions, orientation, slice positions, and spacing before volume creation. Flag inconsistent series rather than sorting only by filename or Instance Number.

5. Large image volumes and Java heap usage.

Mitigation: Use bounded buffers and series-scoped processing; avoid keeping the entire multi-patient dataset in memory and expose configurable temporary/output locations.

6. Filename collisions during normalization.

Mitigation: Use deterministic names containing stable series/file identifiers and maintain a source-to-destination manifest. Never overwrite an existing file without an explicit conflict policy.

7. Native codec portability.

Mitigation: Package and document the required dcm4che runtime components for the supported operating systems. The deliverable will be tested on the stakeholder's target workstation configuration.

11 Expected Impact

The project aims to reduce repetitive pre-processing work in MRI research workflows by combining format validation, directory normalization, conversion, and verification in one local application. The principal contribution is therefore a reliable research-data engineering workflow rather than a replacement for mature medical-image conversion libraries.

A successful prototype will provide a foundation for later extensions such as support for additional DICOM objects, BIDS-compatible metadata export, batch conversion queues, de-identification, richer quality-control reports, and integration with institutional research data-management workflows.

References

- [1] Rorden Lab, *dcm2niix: DICOM to NIfTI converter*, GitHub repository and documentation, current access Aug. 2026.
<https://github.com/rordenlab/dcm2niix>
- [2] Neuroimaging Informatics Technology Initiative (NIfTI), *NIfTI-1 FAQ*, documentation on the NIfTI-1 header and magic field.
<https://nifti.nimh.nih.gov/nifti-1/documentation/faq.html>
- [3] Neuroimaging Informatics Technology Initiative (NIfTI), *NIfTI-1 Fields*, official header field documentation.
https://nifti.nimh.nih.gov/nifti-1/documentation/nifti1fields/index_html.html
- [4] Neuroimaging Informatics Technology Initiative (NIfTI), *Recommended usage of qform and sform*, documentation on spatial transforms.
https://nifti.nimh.nih.gov/nifti-1/documentation/nifti1fields/nifti1fields_pages/qform_usage.html
- [5] DICOM Standards Committee, *DICOM PS3.10: Media Storage and File Format for Media Interchange*, current DICOM standard.
https://dicom.nema.org/medical/dicom/current/output/chtml/part10/chapter_7.html
- [6] dcm4che Project, *dcm4che DICOM Toolkit & Library*, GitHub repository and documentation, current access Aug. 2026.
<https://github.com/dcm4che/dcm4che>